Bootcamp Desarrollo de aplicaciones Fullstack Python Trainee

Evaluación del módulo 5: Fundamentos de bases de datos relacionales

Proyecto: Base de datos AlkeWallet

Elizabeth Rhenals Vergara

1. Introducción.

Como parte de un equipo de programadores recibiste el pedido de Alke Wallet de desarrollar el diseño de una base de datos relacional para gestionar un sistema de una Wallet la cual permitirá a sus usuarios almacenar y gestionar sus fondos, realizar transacciones y ver el historial de transacciones. Nuestro líder técnico ya cuenta con los requerimientos desagregados en un backlog de tareas listo para que comencemos la etapa de desarrollo.

2. Objetivo.

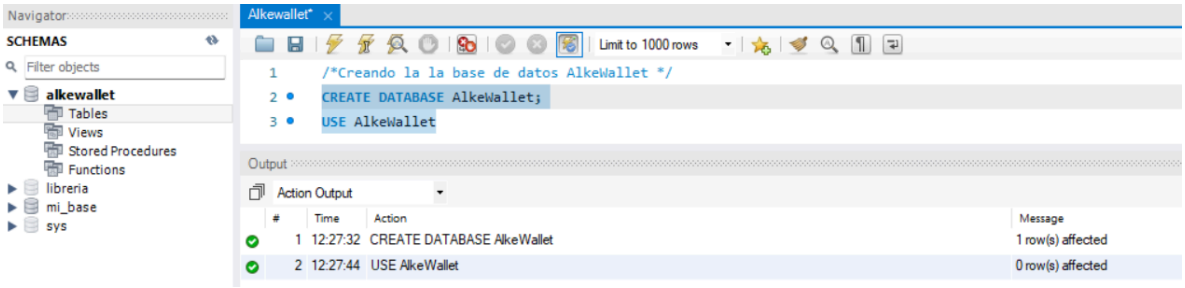
El objetivo de esta evaluación es diseñar el modelo conceptual, definir las relaciones entre las entidades de una wallet y crear la base de datos para almacenar la información que sea necesaria.

3. Metodología.

3.1 Creación de la base de datos

Se creó la base de datos con nombre Alkewallet utilizando MySQL Workbench 8.0 CE (figura 1). Se escribió el código ‘use Alekewallet’ para posicionarse dentro de la base de datos creada correctamente y, de esta manera, empezar a crear las tablas y generar las consultas necesarias.

Figura 1. Captura de la respuesta en la creación de la base de datos Alkewallet

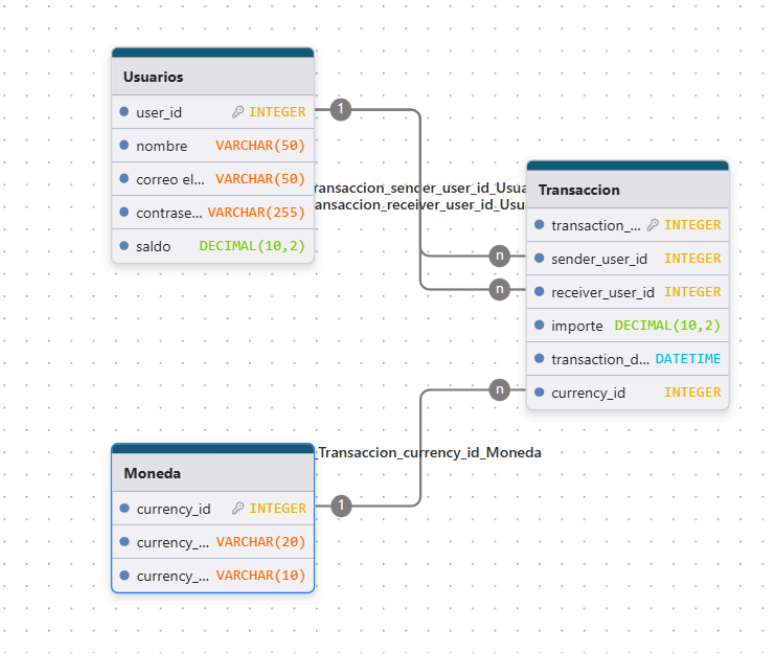


Fuente: Elaboración propia.

3.2 Elaboración del diagrama ER

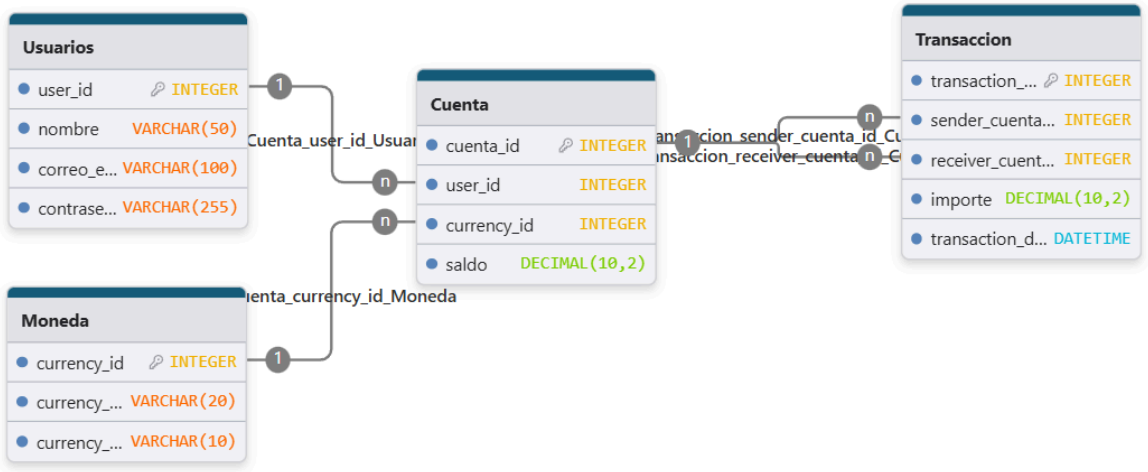
En primer lugar, se realizó un diagrama base con las entidades y atributos propuestos en el proyecto. Se establecieron las relaciones, tipos de datos y restricciones que se observan en la figura 2.

Figura 2. Diagrama ER base con tablas iniciales



Fuente: Elaboración propia.

Sin embargo, se observó que existían incompatibilidades con las 3FN dado que el atributo ‘saldo’ no debería encontrarse dentro de la tabla usuario, por lo que se decidió agregar una nueva tabla que represente el funcionamiento real de una wallet digital (figura 3). De esta manera, ‘Usuarios´y ‘Transacciones’ no tienen una relación directa sino que se relacionan mediante una tabla ‘Cuenta’.



Fuente: Elaboración propia.

Cardinalidades y Opcionalidades del modelo:

El modelo de datos propuesto se compone de las entidades Usuarios, Cuenta, Moneda y Transacción:

- Relación entre Usuarios y Cuenta

Cardinalidad: Un usuario puede tener una o varias cuentas (1:N), mientras que cada cuenta pertenece a un único usuario.

Opcionalidad: La participación de Cuenta es obligatoria, ya que no puede existir una cuenta sin un usuario asociado. La participación de Usuarios es opcional, pues un usuario puede existir sin tener ninguna cuenta registrada.

- Relación entre Moneda y Cuenta

Cardinalidad: Una moneda puede estar asociada a muchas cuentas (1:N), y cada cuenta tiene asociada una sola moneda.

Opcionalidad: La participación de Cuenta es obligatoria, toda cuenta debe tener una moneda asignada. La participación de Moneda es opcional, ya que puede existir una moneda registrada sin que haya cuentas asociadas.

- Relación entre Cuenta y Transacción como emisor

Cardinalidad: Una cuenta puede realizar muchas transacciones como emisor (1:N), mientras que cada transacción tiene exactamente una cuenta emisora.

Opcionalidad: La participación de Transacción es obligatoria, ya que toda transacción requiere una cuenta emisora. La participación de Cuenta es opcional, porque puede existir una cuenta que nunca haya realizado transacciones.

- Relación entre Cuenta y Transacción como receptor

Cardinalidad: Una cuenta puede recibir muchas transacciones (1:N), mientras que cada transacción tiene exactamente una cuenta receptora

Opcionalidad: La participación de Transacción es obligatoria, toda transacción requiere una cuenta receptora. La participación de Cuenta es opcional, pues puede existir una cuenta que nunca haya recibido transacciones.

3.3 Normalización de tablas (3FN)

Se realizó la normalización garantizando la ausencia de redundancias y dependencias transitivas, y asegurando la integridad de la información.

Primera Forma Normal (1FN)

- Todos los atributos de cada entidad contienen valores **atómicos**, es decir, indivisibles.
- No existen grupos repetitivos o listas dentro de un mismo campo**:**
- La entidad **Usuarios** contiene nombre, correo_electronico y contraseña como atributos atómicos.
- La entidad **Cuenta** contiene saldo y currency_id, también indivisibles.
- La entidad **Transacción** almacena importe, transaction_date, sender_cuenta_id y receiver_cuenta_id como valores atómicos.
- La entidad **Moneda** contiene currency_name y currency_symbol de forma indivisible.

Segunda Forma Normal (2FN)

- Todos los atributos no clave dependen completamente de la clave primaria.
- En **Usuarios**, todos los atributos dependen directamente de user_id.

- En **Cuenta**, user_id, currency_id y saldo dependen directamente de cuenta_id.
- En **Transacción**, todos los atributos (sender_cuenta_id, receiver_cuenta_id, importe, transaction_date) dependen únicamente de transaction_id.
- En **Moneda**, currency_name y currency_symbol dependen directamente de currency_id.

Tercera Forma Normal (3FN)

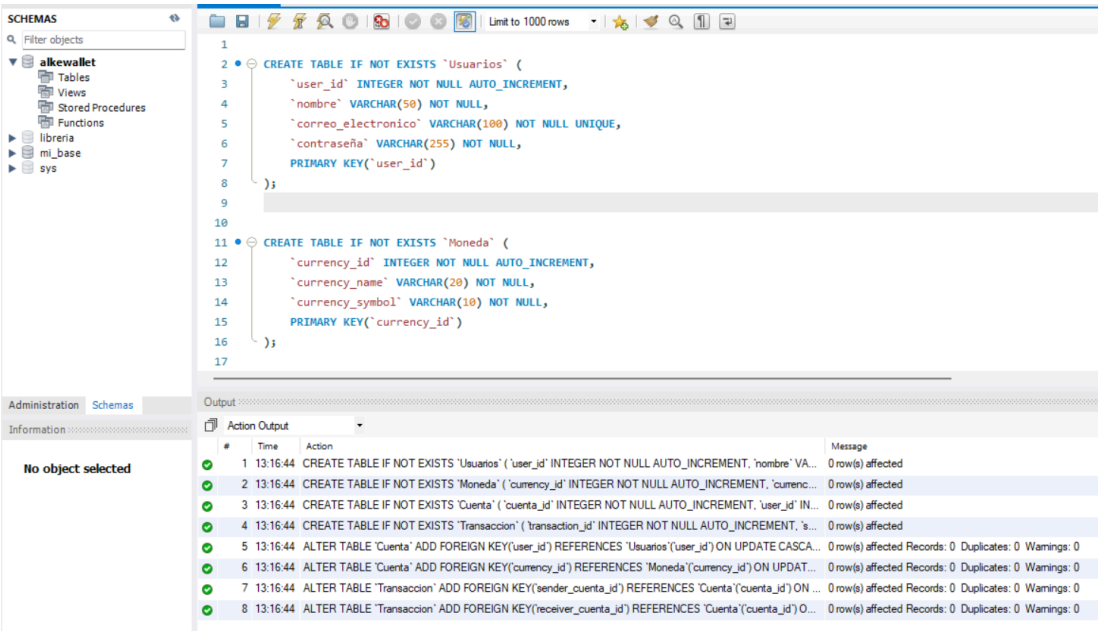
- Ningún atributo no clave depende de otro atributo no clave (**no existen dependencias transitivas**).
- En **Usuarios**, no existen atributos que dependan de otro atributo distinto a la clave primaria.
- En **Cuenta**, saldo depende únicamente de cuenta_id y no de user_id ni de currency_id.
- En **Transacción**, todos los atributos dependen directamente de transaction_id, sin depender de otras columnas como sender_cuenta_id o receiver_cuenta_id.
- En **Moneda**, los atributos dependen únicamente de currency_id.

El script DDL se encuentra en el **ANEXO 1.**

3.4 Creación de tablas en la base de datos

Se empleó el código DDL en la consola de MySQL Workbench 8.0 CE dentro alkewallet (figura 4), donde se generaron correctamente todas las tablas del modelo propuesto.

Figura 4. Captura de la respuesta al código DDL en Workbench

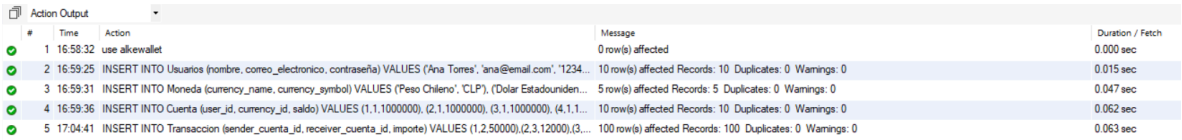


Fuente: Elaboración propia

4. Resultados de las consultas

Teniendo las tablas creadas dentro de la base de datos, se procedió a ingresar datos ejemplo a cada una de las tablas **(ANEXO 2)** para posteriormente realizar las consultas en base a lo aprendido.

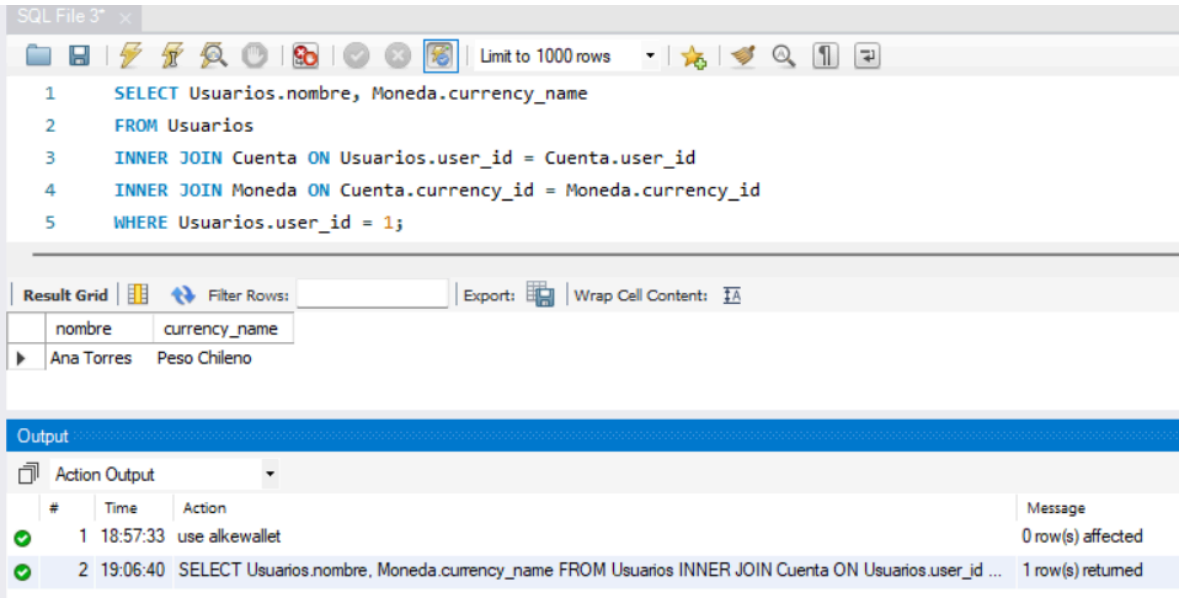
Figura 5. Output del insert de datos en las tablas.



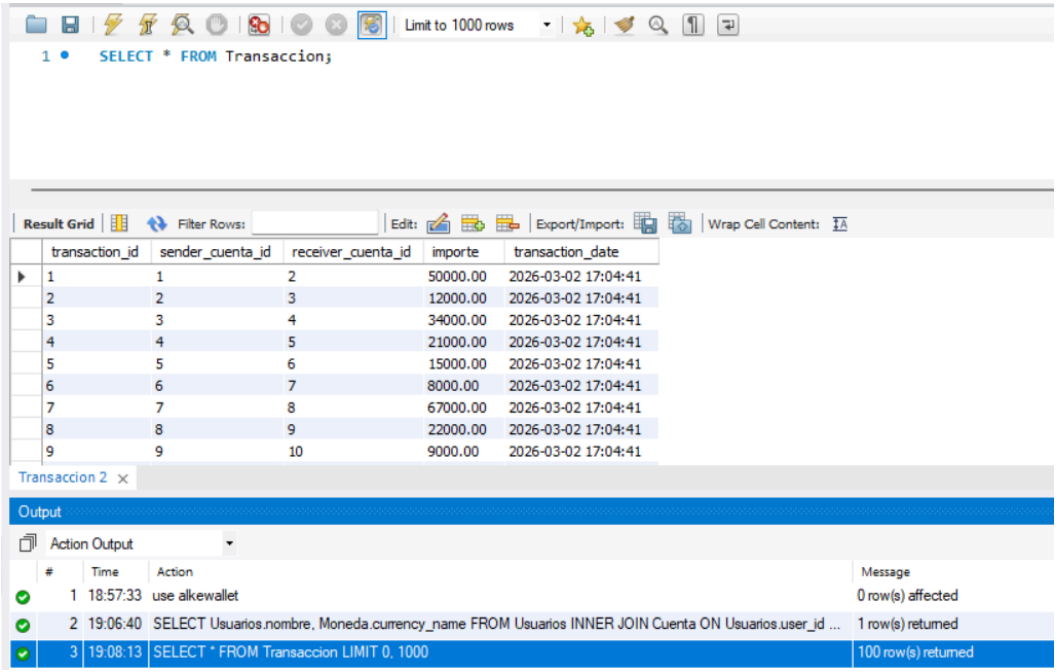
Fuente: Elaboración propia.

- Consulta para obtener el nombre de la moneda elegida por un usuario específico.

Para llevar a cabo esta consulta, partimos de la suposición de que queremos hacerla sobre el usuario con user_id=1



- Consulta para obtener todas las transacciones registradas.



- Consulta para obtener todas las transacciones realizadas por un usuario específico.

```sql
SELECT
Transaccion.transaction_id,
Usuarios.nombre,
Transaccion.importe,
Transaccion.transaction_date
FROM Transaccion
INNER JOIN Cuenta ON Transaccion.sender_cuenta_id = Cuenta.cuenta_id
INNER JOIN Usuarios ON Cuenta.user_id = Usuarios.user_id
WHERE Usuarios.user_id = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

transaction_id	nombre	importe	transaction_date
1	Ana Torres	50000.00	2026-03-02 17:04:41
20	Ana Torres	52000.00	2026-03-02 17:04:41
21	Ana Torres	11000.00	2026-03-02 17:04:41
40	Ana Torres	35000.00	2026-03-02 17:04:41
41	Ana Torres	8000.00	2026-03-02 17:04:41
60	Ana Torres	19000.00	2026-03-02 17:04:41
61	Ana Torres	42000.00	2026-03-02 17:04:41
80	Ana Torres	48000.00	2026-03-02 17:04:41
90	Ana Torres	39000.00	2026-03-02 17:04:41
99	Ana Torres	50000.00	2026-03-02 17:04:41

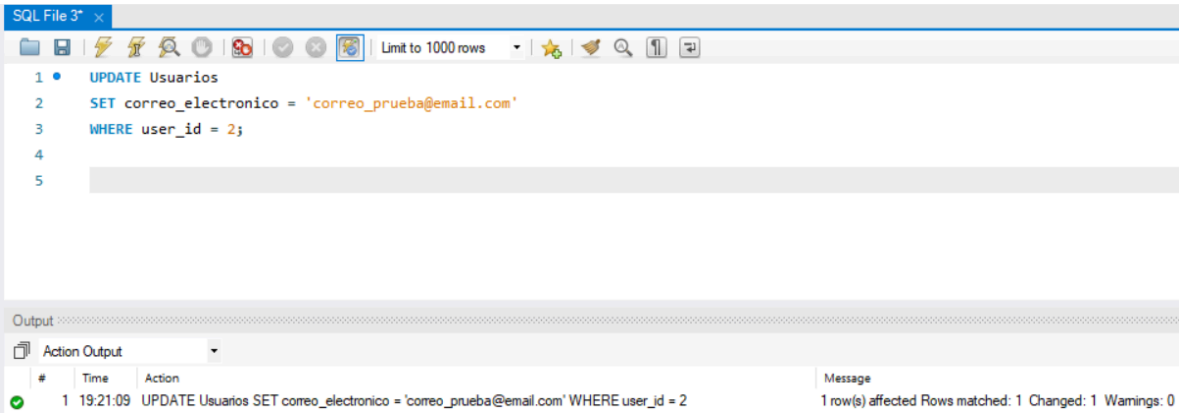
Result 3

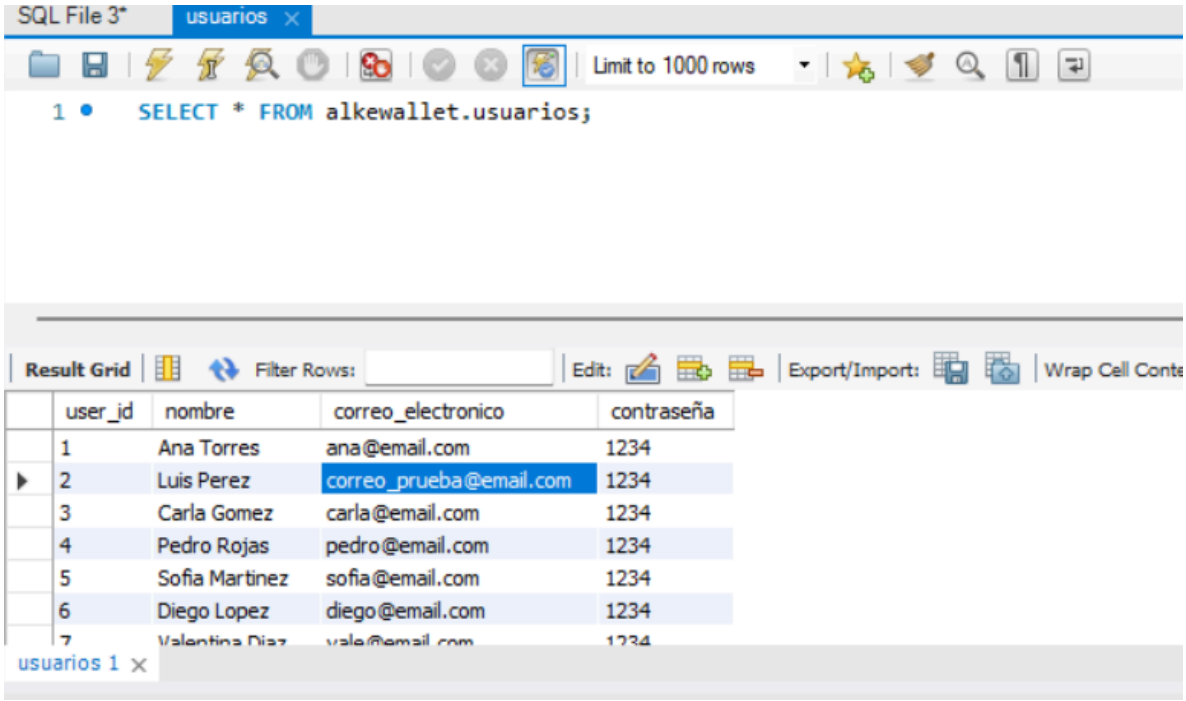
Output

Action Output

#	Time	Action	Message
1	19:13:09	SELECT Transaccion.transaction_id, Usuarios.nombre, Transaccion.importe, Transaccion.transaction_date FR...	10 row(s) returned

- Sentencia DML para modificar el campo correo electrónico de un usuario específico.





user_id	nombre	correo_electronico	contraseña
1	Ana Torres	ana@email.com	1234
2	Luis Perez	correo_prueba@email.com	1234
3	Carla Gomez	carla@email.com	1234
4	Pedro Rojas	pedro@email.com	1234
5	Sofia Martinez	sofia@email.com	1234
6	Diego Lopez	diego@email.com	1234
7	Valentina Diaz	vale@email.com	1234

- Sentencia para eliminar los datos de una transacción (eliminado de la fila completa)

SQL File 3*

Limit to 1000 rows

```
1 • select * from transaccion;
2 • DELETE FROM Transaccion
3   WHERE transaction_id = 1;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

transaction_id	sender_cuenta_id	receiver_cuenta_id	importe	transaction_date
1	1	2	50000.00	2026-03-02 17:04:41
2	2	3	12000.00	2026-03-02 17:04:41
3	3	4	34000.00	2026-03-02 17:04:41
4	4	5	21000.00	2026-03-02 17:04:41
5	5	6	15000.00	2026-03-02 17:04:41
6	6	7	8000.00	2026-03-02 17:04:41
7	7	8	67000.00	2026-03-02 17:04:41

transaccion 6

Output

Action Output

#	Time	Action	Message
1	19:24:35	select * from transaccion LIMIT 0, 1000	100 row(s) returned

SQL File 3*

Limit to 1000 rows

```
1 • select * from transaccion;
2 • DELETE FROM Transaccion
3   WHERE transaction_id = 1;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

transaction_id	sender_cuenta_id	receiver_cuenta_id	importe	transaction_date
2	2	3	12000.00	2026-03-02 17:04:41
3	3	4	34000.00	2026-03-02 17:04:41
4	4	5	21000.00	2026-03-02 17:04:41
5	5	6	15000.00	2026-03-02 17:04:41
6	6	7	8000.00	2026-03-02 17:04:41
7	7	8	67000.00	2026-03-02 17:04:41

transaccion 7

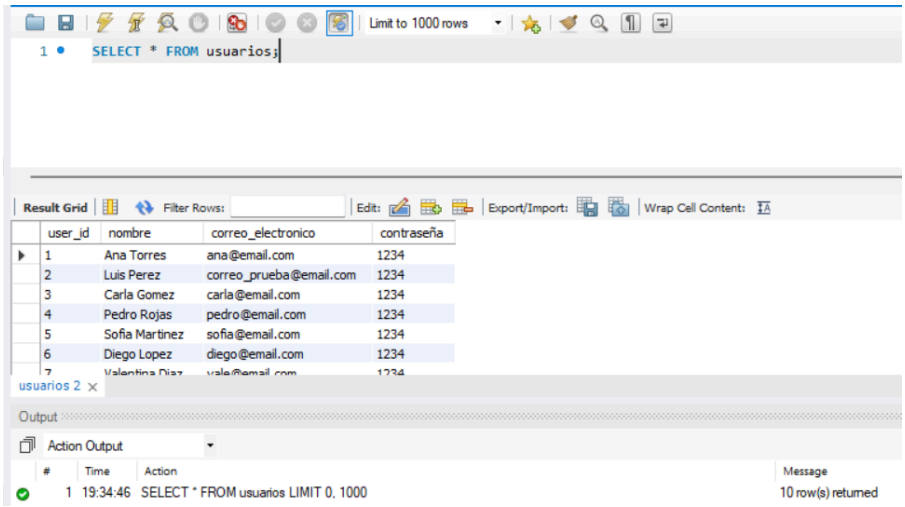
Output

Action Output

#	Time	Action	Message
1	19:24:35	select * from transaccion LIMIT 0, 1000	100 row(s) returned
2	19:25:50	DELETE FROM Transaccion WHERE transaction_id = 1	1 row(s) affected
3	19:25:54	select * from transaccion LIMIT 0, 1000	99 row(s) returned

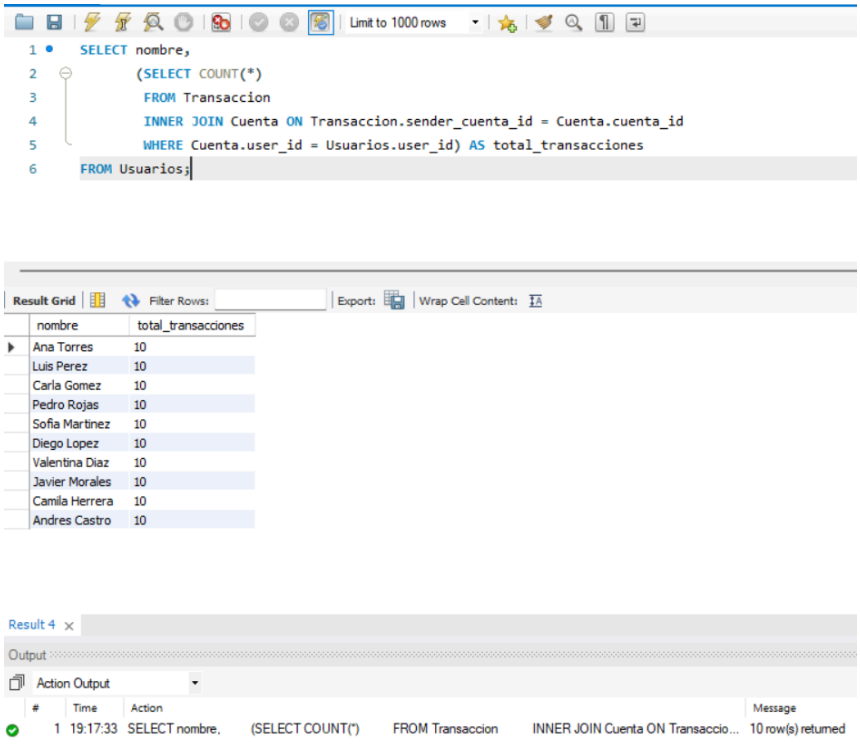
5. Consultas adicionales

- Consulta básica sobre tabla usuarios



user_id	nombre	correo_electronico	contraseña
1	Ana Torres	ana@email.com	1234
2	Luis Perez	correo_prueba@email.com	1234
3	Carla Gomez	carla@email.com	1234
4	Pedro Rojas	pedro@email.com	1234
5	Sofia Martinez	sofia@email.com	1234
6	Diego Lopez	diego@email.com	1234

- Consulta total de transacciones por usuarios



nombre	total_transacciones
Ana Torres	10
Luis Perez	10
Carla Gomez	10
Pedro Rojas	10
Sofia Martinez	10
Diego Lopez	10
Valentina Diaz	10
Javier Morales	10
Camila Herrera	10
Andres Castro	10

- Top 5 usuarios de mayor saldo

SQL File 3* | cuenta

Limit to 1000 rows

```sql
SELECT Usuarios.nombre, SUM(Cuenta.saldo) AS saldo_total
FROM Usuarios
INNER JOIN Cuenta ON Usuarios.user_id = Cuenta.user_id
GROUP BY Usuarios.nombre
ORDER BY saldo_total DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

nombre	saldo_total
Camila Herrera	1030000.00
Sofia Martinez	1020000.00
Javier Morales	1015000.00
Andres Castro	1000000.00
Carla Gomez	1000000.00

- Actualizar saldo de usuarios (simulación)

5	19:37:50	UPDATE Cuenta SET saldo = saldo - 10000 WHERE cuenta_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
6	19:37:50	UPDATE Cuenta SET saldo = saldo + 15000 WHERE cuenta_id = 8	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
7	19:37:50	UPDATE Cuenta SET saldo = saldo + 20000 WHERE cuenta_id = 5	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
8	19:37:50	UPDATE Cuenta SET saldo = saldo - 30000 WHERE cuenta_id = 6	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
9	19:37:50	UPDATE Cuenta SET saldo = saldo + 30000 WHERE cuenta_id = 9	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
10	19:38:06	SELECT * FROM alkewallet.cuenta LIMIT 0, 1000	10 row(s) returned

```sql
SELECT * FROM alkewallet.cuenta;
```

esult Grid | Filter Rows: | Edit:

cuenta_id	user_id	currency_id	saldo
1	1	1	990000.00
2	2	1	1000000.00
3	3	1	1000000.00
4	4	1	1000000.00
5	5	1	1020000.00
6	6	1	970000.00
7	7	1	1000000.00
8	8	1	1015000.00
9	9	1	1030000.00
10	10	1	1000000.00
NULL	NULL	NULL	NULL

- Transacción controlada con ACID y manejo de errores

SQL File 3* | cuenta

Limit to 1000 rows

```sql
START TRANSACTION;

UPDATE Cuenta SET saldo = saldo - 5000 WHERE cuenta_id = 1;
UPDATE Cuenta SET saldo = saldo + 5000 WHERE cuenta_id = 2;

COMMIT;

INSERT INTO Transaccion (sender_cuenta_id, receiver_cuenta_id, importe)
VALUES (999, 1, 5000);
```

Output

Action Output

#	Time	Action	Message
1	19:43:35	START TRANSACTION	0 row(s) affected
2	19:43:35	UPDATE Cuenta SET saldo = saldo - 5000 WHERE cuenta_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
3	19:43:35	UPDATE Cuenta SET saldo = saldo + 5000 WHERE cuenta_id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
4	19:43:35	COMMIT	0 row(s) affected
5	19:43:35	INSERT INTO Transaccion (sender_cuenta_id, receiver_cuenta_id, importe) VALUES (999, 1, 5000)	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (`alkewallet`.`transaccion`, CON...

- Modificar tabla Usuarios agregando fecha de creacion

SQL File 3*

Limit to 1000 rows

```sql
ALTER TABLE Usuarios
ADD fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP;

DESCRIBE Usuarios;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
user_id	int	NO	PRI	NULL	auto_increment
nombre	varchar(50)	NO		NULL	
correo_electronico	varchar(100)	NO	UNI	NULL	
contraseña	varchar(255)	NO		NULL	
fecha_creacion	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Result Grid | Filter Rows: | Edit: | Export/Import:

user_id	nombre	correo_electronico	contraseña	fecha_creacion
2	Luis Perez	correo_prueba@email.com	1234	2026-03-02 19:55:40
3	Carla Gomez	carla@email.com	1234	2026-03-02 19:55:40
4	Pedro Rojas	pedro@email.com	1234	2026-03-02 19:55:40
5	Sofia Martinez	sofia@email.com	1234	2026-03-02 19:55:40
6	Diego Lopez	diego@email.com	1234	2026-03-02 19:55:40
7	Valentina Diaz	vale@email.com	1234	2026-03-02 19:55:40
8	Javier Morales	javier@email.com	1234	2026-03-02 19:55:40
9	Camila Herrera	camila@email.com	1234	2026-03-02 19:55:40
10	Andres Castro	andres@email.com	1234	2026-03-02 19:55:40
NULL	NULL	NULL	NULL	NULL

ANEXO 1. Script DDL

```
CREATE TABLE IF NOT EXISTS `Usuarios` (
	`user_id` INTEGER NOT NULL AUTO_INCREMENT,
	`nombre` VARCHAR(50) NOT NULL,
	`correo_electronico` VARCHAR(100) NOT NULL UNIQUE,
	`contraseña` VARCHAR(255) NOT NULL,
	PRIMARY KEY(`user_id`)
);

CREATE TABLE IF NOT EXISTS `Moneda` (
	`currency_id` INTEGER NOT NULL AUTO_INCREMENT,
	`currency_name` VARCHAR(20) NOT NULL,
	`currency_symbol` VARCHAR(10) NOT NULL,
	PRIMARY KEY(`currency_id`)
);

CREATE TABLE IF NOT EXISTS `Cuenta` (
	`cuenta_id` INTEGER NOT NULL AUTO_INCREMENT,
	`user_id` INTEGER NOT NULL,
	`currency_id` INTEGER NOT NULL,
	`saldo` DECIMAL(10,2) NOT NULL CHECK(`saldo` >= 0),
	PRIMARY KEY(`cuenta_id`)
);
```

```
CREATE TABLE IF NOT EXISTS `Transaccion` (
	`transaction_id` INTEGER NOT NULL AUTO_INCREMENT,
	`sender_cuenta_id` INTEGER NOT NULL,
	`receiver_cuenta_id` INTEGER NOT NULL,
	`importe` DECIMAL(10,2) NOT NULL,
	`transaction_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
	PRIMARY KEY(`transaction_id`)
);

ALTER TABLE `Cuenta`
ADD FOREIGN KEY(`user_id`) REFERENCES `Usuarios`(`user_id`)
ON UPDATE CASCADE ON DELETE RESTRICT;
ALTER TABLE `Cuenta`
ADD FOREIGN KEY(`currency_id`) REFERENCES `Moneda`(`currency_id`)
ON UPDATE CASCADE ON DELETE RESTRICT;
ALTER TABLE `Transaccion`
ADD FOREIGN KEY(`sender_cuenta_id`) REFERENCES `Cuenta`(`cuenta_id`)
ON UPDATE CASCADE ON DELETE RESTRICT;
ALTER TABLE `Transaccion`
ADD FOREIGN KEY(`receiver_cuenta_id`) REFERENCES `Cuenta`(`cuenta_id`)
ON UPDATE CASCADE ON DELETE RESTRICT;
```

ANEXO 2. Códigos de insert en las tablas

```
INSERT INTO Usuarios (nombre, correo_electronico, contraseña) VALUES
('Ana Torres', 'ana@email.com', '1234'),
('Luis Perez', 'luis@email.com', '1234'),
('Carla Gomez', 'carla@email.com', '1234'),
('Pedro Rojas', 'pedro@email.com', '1234'),
('Sofia Martinez', 'sofia@email.com', '1234'),
('Diego Lopez', 'diego@email.com', '1234'),
('Valentina Diaz', 'vale@email.com', '1234'),
('Javier Morales', 'javier@email.com', '1234'),
('Camila Herrera', 'camila@email.com', '1234'),
('Andres Castro', 'andres@email.com', '1234');

INSERT INTO Moneda (currency_name, currency_symbol) VALUES
('Peso Chileno', 'CLP'),
('Dolar Estadounidense', 'USD'),
('Euro', 'EUR'),
('Bitcoin', 'BTC'),
('Peso Argentino', 'ARS');

INSERT INTO Cuenta (user_id, currency_id, saldo) VALUES
(1,1,1000000),
(2,1,1000000),
```

(3,1,1000000),

(4,1,1000000),

(5,1,1000000),

(6,1,1000000),

(7,1,1000000),

(8,1,1000000),

(9,1,1000000),

(10,1,1000000);

INSERT INTO Transaccion (sender_cuenta_id, receiver_cuenta_id, importe) VALUES

(1,2,50000),(2,3,12000),(3,4,34000),(4,5,21000),(5,6,15000),

(6,7,8000),(7,8,67000),(8,9,22000),(9,10,9000),(10,1,30000),

(2,5,45000),(3,6,17000),(4,7,26000),(5,8,31000),(6,9,29000),

(7,10,41000),(8,1,18000),(9,2,24000),(10,3,36000),(1,4,52000),

(1,5,11000),(2,6,25000),(3,7,33000),(4,8,47000),(5,9,15000),

(6,10,22000),(7,1,37000),(8,2,14000),(9,3,6000),(10,4,28000),

(2,7,39000),(3,8,44000),(4,9,12500),(5,10,21500),(6,1,30000),

(7,2,18000),(8,3,27500),(9,4,49000),(10,5,16000),(1,6,35000),

(1,7,8000),(2,8,23000),(3,9,54000),(4,10,21000),(5,1,12000),

(6,2,26000),(7,3,33000),(8,4,41000),(9,5,9000),(10,6,37000),

(2,9,14500),(3,10,29500),(4,1,47000),(5,2,16000),(6,3,38000),

(7,4,20000),(8,5,15000),(9,6,34000),(10,7,28000),(1,8,19000),

(1,9,42000),(2,10,17500),(3,1,36000),(4,2,22000),(5,3,31000),

(6,4,45000),(7,5,29000),(8,6,13000),(9,7,24000),(10,8,50000),

(2,1,17000),(3,2,26000),(4,3,38000),(5,4,9000),(6,5,15000),

(7,6,34000),(8,7,27000),(9,8,19000),(10,9,21000),(1,10,48000),

(2,3,22000),(3,4,14000),(4,5,35000),(5,6,28000),(6,7,16000),

(7,8,47000),(8,9,12500),(9,10,33000),(10,1,26000),(1,2,39000),

(3,5,17500),(4,6,21500),(5,7,29000),(6,8,31000),(7,9,18000),

(8,10,42000),(9,1,23000),(10,2,36000),(1,3,50000),(2,4,27000);